ean universidad

acreditada
en alta calidad
res. N° 023654 del Mineducación
10/12/21, vigencia 10/12/27

PYTHON PARA PROCESAMIENTO, CALIDAD Y AUTOMATIZACIÓN DE DATOS

Sesión 2 · Limpieza y transformación de datos

De un archivo sucio a una tabla confiable

Jawy Andrés Romero Pinto · Miércoles 4 de noviembre de 2026 · 08:00 – 10:00 a. m.

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Acreditaciones:

Agenda de hoy

Miércoles 4 de noviembre · 08:00 – 10:00 a. m.

08:00	**Bienvenida y repaso del reto**	Los problemas que encontraste ayer son la lista de tareas de hoy.
08:12	**Principios de limpieza**	Copia, una línea por decisión, corregir sin borrar, medir.
08:16	**Texto y categorías**	strip, lower, title, tildes y diccionarios de mapeo.
08:26	**Tipos y fechas**	Documentos, salarios con puntos, fechas en dos formatos.
08:34	**Nulos y duplicados**	Marcar, imputar o eliminar; duplicados exactos y por llave.
08:39	**Unir y agregar**	pd.cut, merge (BUSCARV), groupby y pivot_table (tabla dinámica).
08:52	**Antes y después**	Lo que vas a lograr en el laboratorio.
08:55	**Laboratorio**	Limpias afiliados.csv completo y exportas afiliados_limpios.csv (60 min).
09:55	**Cierre y reto**	Lo que te llevas y el reto con las PQRS.

01

Lo que encontramos ayer

El diagnóstico de la Sesión 1 es la lista de tareas de hoy.

Repaso: lo que encontramos ayer

Cada problema se corrige con código, no a mano

Problema	Columna	Evidencia	Técnica de hoy
Mismas ciudades escritas distinto	ciudad	39 valores distintos (deberían ser 15)	`.str` + diccionario de mapeo
Categorías inconsistentes	genero, estado	"F", "f", "Femenino", "ACTIVO", "Activo "	`.str` + `.map()`
Números guardados como texto	salario_base	"3.549.000": 63 no convertibles	`.str.replace` + `to_numeric`
Documentos con puntos y espacios	numero_documento	151 filas con "." o espacios	`.str.replace(regex=True)`
Fechas en dos formatos	fecha_nacimiento	"1971-09-13" y "13/09/1971"	`to_datetime`
Filas repetidas	todas	90 duplicados exactos	`drop_duplicates`

Regla del curso: **cada problema se corrige con código, no a mano.** Así el mes que viene se corrige solo.

Principios de una buena limpieza

Antes de tocar una sola celda



Trabaja sobre una copia

Primero `.copy()`, luego se corrige. El archivo original nunca se toca.



Una decisión, una línea

Cada corrección queda escrita: qué se cambió y por qué.



Corrige, no borres

Borrar filas esconde problemas que el negocio debe conocer.



Mide antes y después

Filas, vacíos y valores distintos. Si no lo mides, no sabes si mejoró.



Lo dudoso se marca

Si no sabes el valor correcto, déjalo vacío (NaN). No lo inventes.

Limpiar **no es maquillar**: el objetivo es que los datos digan la verdad, no que se vean bonitos.

ean universidad

02

Texto y categorías

Las mismas funciones de texto de Python, aplicadas a la columna entera.

Texto en columna: el accesor .str

Una función de texto aplicada a las 3.120 filas de una vez

Qué es

`.str` es la puerta a las funciones de texto de Python (strip, lower, title, replace) para **toda la columna**, sin escribir un ciclo.

Para qué sirve

Unificar espacios, mayúsculas y tildes **antes** de comparar o contar. "Bogotá ", "bogota" y "BOGOTA" deben ser la misma cosa.

Analogía con Excel

Es ESPACIOS() o NOMPROPIO() en la celda C2, arrastrada hasta la fila 3.120. Aquí no se arrastra: la columna es una sola cosa.

Cómo manejarlo

Encadena de izquierda a derecha: strip, luego lower o title. Mira `value_counts()` antes y después. Reemplaza la columna solo cuando el resultado te convence.

Abstracción desde el código

```
df["ciudad"].str.strip()
df["ciudad"].str.strip().str.title()


columna.str.funcion()
# una función, todas las filas
```

La columna es el objeto y `.str` es su caja de herramientas de texto. Tú no escribes el ciclo: pandas lo ejecuta por ti en C, fila por fila.

Limpiar texto con .str

strip, lower, upper, title: se encadenan de izquierda a derecha

[1]
```python
df = afiliados_raw.copy()                    # nunca sobre el original
print(repr(df.loc[2, "nombres"]))            # como viene en el archivo

df["nombres"] = df["nombres"].str.strip().str.title()
df["apellidos"] = df["apellidos"].str.strip().str.title()
print(repr(df.loc[2, "nombres"]))            # después

# Quitar tildes: "Medellín" → "Medellin"
import unicodedata
def quitar_tildes(texto):
    if pd.isna(texto):
        return texto
    texto = unicodedata.normalize("NFD", texto)
    return "".join(c for c in texto if unicodedata.category(c) != "Mn")

print(quitar_tildes("Bogotá D.C."), "|", "  BOGOTÁ ".strip().title())
```

```
'edwin héctor'
'Edwin Héctor'
Bogota D.C. | Bogotá
```

Qué observar

- `.str` aplica la función a **toda la columna**.
- `.strip()` quita espacios; `.title()` pone Tipo Título.
- `quitar_tildes` deja "Bogota D.C.": sirve para comparar, no para mostrar.

`.str.title()` es útil para nombres, pero cuidado con siglas ("D.C." o "NIT"). Por eso las **categorías no se arreglan con title: se mapean**.

Categorías: el diccionario de mapeo

Una tabla de equivalencias escrita como dato, no como código

Qué es

Un diccionario `{"como llega": "como debe quedar"}`. `.map()` lo aplica a toda la columna y devuelve el valor oficial de cada fila.

Para qué sirve

Convertir 11 formas de escribir 2 géneros en **exactamente 2 valores válidos**, y que lo inesperado quede a la vista.

Analogía con Excel

La tabla de homologación que pones al lado para un BUSCARV: a la izquierda lo que llega, a la derecha el valor oficial.

Cómo manejarlo

Construye el diccionario mirando `sorted(col.unique())`. Usa `map` si el conjunto válido es cerrado; `replace` si solo corriges algunos. Cuenta los NaN que quedan: son lo que no previste.

Abstracción desde el código

```
MAPA = {"f": "F", "femenino": "F",
        "m": "M", "masculino": "M"}


clave =
df["genero"].str.strip().str.lower()
df["genero"] = clave.map(MAPA)


df["genero"].isna().sum()  # lo que no
estaba
```

El conocimiento del negocio ("B/quilla es Barranquilla") no se esconde en el código: vive en un dato que cualquiera puede leer, discutir y corregir sin saber programar.

Normalizar categorías con diccionarios

Un diccionario de mapeo es la tabla de equivalencias del negocio

[2]
```python
print(sorted(df["genero"].dropna().unique()), "+ vacíos")

# 1) uniformar  2) traducir con un diccionario
mapa_genero = {"F": "F", "FEMENINO": "F", "M": "M", "MASCULINO": "M"}
df["genero"] = df["genero"].str.strip().str.upper().map(mapa_genero)
print(df["genero"].value_counts(dropna=False).to_dict())

# .replace cambia solo lo indicado y deja el resto igual
df["estado"] = (df["estado"].str.strip().str.capitalize()
                  .replace({"Pensionada": "Pensionado"}))
df["estado"].value_counts()
```

```
['F', 'F ', 'FEMENINO', 'Femenino', 'M', 'M ', 'MASCULINO', 'Masculino', 'f', 'm']
+ vacíos
{'F': 1649, 'M': 1407, nan: 64}
estado
Activo        1955
Inactivo       583
Pensionado     582
Name: count, dtype: int64
```

Qué observar

- 11 formas de escribir 2 géneros.
- `.map(diccionario)`: lo que **no está** en el mapa queda NaN. Así detectas lo inesperado.
- `.replace({...})` cambia solo lo indicado y deja el resto igual.

map vs replace: map es estricto (todo debe estar en el diccionario); replace es permisivo. Usa map cuando el conjunto de valores válidos es cerrado.

El caso ciudad: 39 formas, 15 ciudades

Primero una clave limpia, luego el diccionario

[3]
```
print("Antes:", df["ciudad"].nunique(), "ciudades distintas")
clave = (df["ciudad"].str.strip().str.lower()
           .map(quitar_tildes).str.replace(".", "", regex=False))
print(len(clave.unique()), "claves:", sorted(clave.unique())[:6], "...")

mapa_ciudad = {"bogota": "Bogotá D.C.", "bogota dc": "Bogotá D.C.",
"medellin": "Medellín",
    "cali": "Cali", "santiago de cali": "Cali", "barranquilla":
"Barranquilla",
    "b/quilla": "Barranquilla", "cartagena": "Cartagena", "cartagena de
indias": "Cartagena",
    "bucaramanga": "Bucaramanga", "b/manga": "Bucaramanga", "cucuta":
"Cúcuta",
    "ibague": "Ibagué", "pereira": "Pereira", "manizales": "Manizales",
"tunja": "Tunja",
    "villavicencio": "Villavicencio", "pasto": "Pasto", "neiva": "Neiva",
"santa marta": "Santa Marta"}
df["ciudad"] = clave.map(mapa_ciudad)          # lo que no está en el mapa →
NaN
print("Después:", df["ciudad"].nunique(), "ciudades | sin mapear:",
df["ciudad"].isna().sum())
df["ciudad"].value_counts().head(5)
```

```
Antes: 39 ciudades distintas
20 claves: ['b/manga', 'b/quilla', 'barranquilla', 'bogota', 'bogota dc',
'bucaramanga'] ...
Después: 15 ciudades | sin mapear: 0
ciudad
Bogotá D.C.     934
Medellín        408
Cali            329
Barranquilla    248
Bucaramanga     170
Name: count, dtype: int64
```

Qué observar

- La **clave** normaliza: sin espacios, minúsculas, sin tildes ni puntos. De 39 a 20.
- El diccionario hace el resto: 20 claves, 15 ciudades.
- Verifica siempre: `nunique()` = 15 y `isna().sum()` = 0.

El computador no sabe que "B/quilla" es Barranquilla. Esa equivalencia es **conocimiento del negocio** y vive en el diccionario, donde cualquiera puede revisarla.

ean universidad

03

Tipos y fechas

Documentos, salarios con puntos de miles y fechas en dos formatos.

Tipos de datos: número, texto o código

pandas adivina el tipo al leer; tú decides el correcto

Qué es

Cada columna tiene un tipo (`dtype`): texto (object), entero, decimal, fecha. `info()` lo muestra. Un "3.549.000" con puntos es texto aunque parezca número.

Para qué sirve

Solo con el tipo correcto se puede sumar, promediar, ordenar o restar. Con texto, `mean()` falla o da basura.

Analogía con Excel

En Excel el número guardado como texto se alinea a la izquierda y SUMA() lo ignora en silencio. Aquí pasa igual, pero `info()` te avisa.

Cómo manejarlo

Pregunta: **¿lo voy a sumar?** Si no (documento, teléfono, código DANE), es texto: `dtype=str`. Si sí: quita puntos, `to_numeric(errors="coerce")` y cuenta los NaN nuevos.

Abstracción desde el código

```
df = pd.read_csv("afiliados.csv",
        dtype={"numero_documento": str})


sin_puntos =
df["salario_base"].str.replace(".", "",
regex=False)
df["salario_base"] =
pd.to_numeric(sin_puntos,
                        errors="coerce")
```

`errors="coerce"` es una decisión explícita: "lo que no se pueda convertir, márcalo vacío y sigue". Sin ella el programa se detiene en el primer dato raro. Luego cuentas cuántos vacíos aparecieron.

Documentos y números con formato

Quitar lo que sobra y convertir al tipo correcto

[4]
```python
con_basura = df["numero_documento"].str.contains(r"[.\s]", na=False).sum()
print("Documentos con puntos o espacios:", con_basura)
df["numero_documento"] = df["numero_documento"].str.replace(r"[.\s]", "",
regex=True)

# Salario: "3.549.000" → 3549000
print("Directo:", pd.to_numeric(df["salario_base"],
errors="coerce").isna().sum(), "no convertibles")
sin_puntos = df["salario_base"].str.replace(".", "", regex=False)
df["salario_base"] = pd.to_numeric(sin_puntos, errors="coerce")
print("Sin puntos:", df["salario_base"].isna().sum(), "vacíos |",
df["salario_base"].dtype)

# Regla de negocio: un salario <= 0 no es válido → se marca como vacío
invalidos = df["salario_base"] <= 0
print("Salarios <= 0:", invalidos.sum(), "→ quedan vacíos, no se inventan")
df.loc[invalidos, "salario_base"] = None
```

```
Documentos con puntos o espacios: 151
Directo: 95 no convertibles
Sin puntos: 32 vacíos | float64
Salarios <= 0: 15 → quedan vacíos, no se inventan
```

Qué observar

- `r"[.\s]"`: una expresión regular que significa "punto o espacio".
- `to_numeric` directo falla en 95; sin puntos, solo quedan los **32 vacíos**.
- 15 salarios ≤ 0 se marcan vacíos: **regla de negocio** escrita en código.

¿Es un número o un código? Si **nunca lo vas a sumar ni promediar** (documento, teléfono, código DANE), es texto: léelo con `dtype=str`.

Fechas: de texto a datetime

Un texto "13/09/1971" no sabe qué es un año; una fecha sí

Qué es

`datetime` es un tipo que entiende días, meses y años. Permite restar fechas, extraer el año o el mes y comparar "antes" y "después".

Para qué sirve

Calcular edad, antigüedad y **días de respuesta** de una PQRS; ordenar y agrupar por mes.

Analogía con Excel

En Excel una fecha es un número de serie con formato. Si llega como texto, FECHA() y las restas fallan exactamente igual que aquí.

Cómo manejarlo

Declara cada formato con `format=`; lo que no encaja queda NaT (`errors="coerce"`); combina los intentos con `fillna`. Luego revisa rangos imposibles (1890, 2031).

Abstracción desde el código

```
col = df["fecha_nacimiento"]
iso = pd.to_datetime(col,
    format="%Y-%m-%d", errors="coerce")
latino = pd.to_datetime(col,
    format="%d/%m/%Y", errors="coerce")
df["fecha_nacimiento"] =
iso.fillna(latino)
```

Dos formatos, dos intentos, un resultado. Cada intento es explícito: nunca dejes que la máquina adivine si 03/04 es el 3 de abril o el 4 de marzo.

Fechas: dos formatos en la misma columna

pd.to_datetime convierte texto en fechas de verdad

[5]
```
print(df["fecha_nacimiento"].head(3).tolist())      # dos formatos mezclados

for col in ["fecha_nacimiento", "fecha_afiliacion"]:
    iso    = pd.to_datetime(df[col], format="%Y-%m-%d", errors="coerce")
    latino = pd.to_datetime(df[col], format="%d/%m/%Y", errors="coerce")
    print(col, "| solo ISO:", iso.isna().sum(), "vacíos → con dd/mm/aaaa:",
iso.fillna(latino).isna().sum())
    df[col] = iso.fillna(latino)

# Fechas imposibles: nacidos en el futuro o antes de 1920
malas = (df["fecha_nacimiento"] > CORTE) | (df["fecha_nacimiento"] < "1920-
01-01")
print("Imposibles:", malas.sum(), "→", df.loc[malas,
"fecha_nacimiento"].dt.year.tolist()[:4], "...")
df.loc[malas, "fecha_nacimiento"] = pd.NaT

df["edad"] = ((CORTE - df["fecha_nacimiento"]).dt.days //
365.25).astype("Int64")
df[["fecha_nacimiento", "edad"]].head(3)
```

```
['1971-09-13', '1981-04-14', '1988-06-04']
fecha_nacimiento | solo ISO: 223 vacíos → con dd/mm/aaaa: 0
fecha_afiliacion | solo ISO: 0 vacíos → con dd/mm/aaaa: 0
Imposibles: 13 → [1890, 2031, 1890, 1900] ...
```

	fecha_nacimiento	edad
0	1971-09-13	55
1	1981-04-14	45
2	1988-06-04	38

Qué observar

- `format=` exacto + `errors="coerce"`: lo que no encaja queda NaT, no falla.
- Solo ISO deja 223 vacías; `fillna` con dd/mm/aaaa las recupera **todas**.
- 13 fechas imposibles (1890, 2031) se marcan; con fechas reales ya se calcula la **edad**.

Nunca uses `dayfirst=True` a ciegas con formatos mezclados: 03/04 puede ser 3 de abril o 4 de marzo. Declara cada formato.

La trampa del cero inicial

Por qué un código no es un número

[6]
```
sin_dtype = pd.read_excel("datos/oficinas.xlsx", sheet_name="ciudades")
cat = pd.read_excel("datos/oficinas.xlsx", sheet_name="ciudades",
                    dtype={"codigo_dane": str})

med = sin_dtype["ciudad"] == "Medellín"
print("Sin dtype:", sin_dtype.loc[med, "codigo_dane"].iloc[0], "←",
sin_dtype["codigo_dane"].dtype)
print("Con dtype:", cat.loc[med, "codigo_dane"].iloc[0], "←",
cat["codigo_dane"].dtype)
print("Si ya se perdió el cero:", str(5001).zfill(5))
```

```
Sin dtype: 5001 ← int64
Con dtype: 05001 ← str
Si ya se perdió el cero: 05001
```

Qué observar

- Sin `dtype`, pandas lee 05001 como el entero **5001**.
- El código DANE de Medellín es **05001**: un cruce por código fallaría.
- `.str.zfill(5)` lo recupera si ya se perdió.

Pasa igual en Excel cuando abres un CSV con doble clic. Por eso tantos documentos, teléfonos, NIT y códigos postales llegan "dañados".

ean universidad



04

Nulos y duplicados

Eliminar, imputar o marcar; duplicados exactos y por llave de negocio.

Valores nulos: ¿eliminar, imputar o marcar?

No hay una respuesta única: depende del uso del dato

Estrategia	Código	Cuándo usarla	Riesgo
Marcar (dejar NaN)	no hacer nada	Por defecto. El dato no se conoce.	Ninguno: es honesto
Imputar un texto	`fillna("Sin dato")`	Categorías para reportes y conteos	Bajo
Imputar un número	`fillna(mediana)`	Modelos estadísticos, con criterio y documentado	Altera promedios y totales
Eliminar la fila	`dropna(subset=[...])`	Sin ese campo la fila no sirve (p. ej. sin id)	Pierdes registros

En datos de personas **nunca inventes un salario o una fecha** para llenar el hueco. Márcalo y repórtalo: eso es un hallazgo de calidad (Sesión 3).

Valores nulos: NaN y NaT

La marca de "no se conoce" no es un cero ni un texto vacío

Qué es

`NaN` (número o texto) y `NaT` (fecha) son el valor especial de pandas para **dato desconocido**. `isna()` los detecta; `info()` los cuenta.

Para qué sirve

Que los cálculos no se contaminen: `mean()` ignora los NaN. Con un 0 inventado en el salario, el promedio baja y el aporte esperado se calcula mal.

Analogía con Excel

Una celda vacía frente a un 0 escrito a mano: PROMEDIO() las trata distinto. pandas también, y además te deja contarlas con una línea.

Cómo manejarlo

`isna().sum()` antes de decidir. Categoría para reportes: `fillna("Sin dato")`. Dato de persona (salario, fecha): se queda vacío y se reporta. Fila sin llave: `dropna(subset=[...])`.

Abstracción desde el código

```
df.isna().sum()                    # medir

df["genero"] = df["genero"].fillna("Sin
dato")

df["salario_base"].mean()          # ignora
NaN
df.dropna(subset=["id_afiliado"])
```

El vacío también es información: dice "aquí nadie sabe". La decisión sobre cada columna se escribe en una línea y mañana se puede defender ante el negocio.

Nulos en la práctica

Medir, decidir y documentar

[7]

```
vacios = df.isna().sum()
print(vacios[vacios > 0])

df["genero"] = df["genero"].fillna("Sin dato")   # categoría: se imputa un texto
# salario_base y fechas se quedan vacíos: no se inventan
df["genero"].value_counts()
```

```
numero_documento    15
fecha_nacimiento    13
genero              64
salario_base        47
email               15
telefono            22
edad                13
dtype: int64
genero
F           1649
M           1407
Sin dato      64
Name: count, dtype: int64
```

Qué observar

- `isna().sum()` mide **antes** de decidir.
- `genero`: categoría para reportes, se imputa "Sin dato".
- `salario_base` (47) y fechas (13): se quedan vacíos.

Los 47 salarios vacíos son 32 que venían vacíos + 15 inválidos (≤ 0). **Quedan marcados**, y mañana los contamos como incumplimiento de una regla de calidad.

Duplicados: exactos y por llave de negocio

Qué significa "la misma fila" lo define el negocio, no pandas

Qué es

Filas que representan el **mismo registro**: idénticas en todas las columnas (exactas) o la misma persona con detalles distintos (misma llave: el documento).

Para qué sirve

Un duplicado infla conteos, totales y aportes. 90 filas de más son 90 afiliados que no existen en el informe a la Dirección.

Analogía con Excel

"Quitar duplicados" de Excel te pide marcar columnas. `subset=` es exactamente esa casilla; sin subset, compara todas.

Cómo manejarlo

Primero limpia la llave (documento sin puntos). Luego `duplicated(subset=, keep=False)` para **mirarlos**. Luego `drop_duplicates(keep="first")`. Anota cuántas se fueron.

Abstracción desde el código

```
df.duplicated().sum()       # exactos: 90

rep = df.duplicated(
    subset="numero_documento", keep=False)
df[rep]                     # mirarlos

df = df.drop_duplicates(
    subset="numero_documento")
```

La llave de negocio define qué es "lo mismo". Esa decisión es del negocio; pandas solo la ejecuta. Y el orden importa: limpiar la llave antes de buscar repetidos.

Duplicados: exactos y por llave

No todo duplicado se ve igual

[8]
```python
print("Filas:", len(df), "| duplicados exactos:", df.duplicated().sum())
df = df.drop_duplicates()

# Mismo documento en otra fila (el documento ya está sin puntos)
dup = df.duplicated(subset="numero_documento", keep="first") &
df["numero_documento"].notna()
print("Duplicados por documento:", dup.sum())
repetidos = df["numero_documento"].isin(df.loc[dup, "numero_documento"])
muestra = df[repetidos].sort_values("numero_documento")[
    ["id_afiliado", "numero_documento", "nombres", "apellidos"]].head(4)

df = df[~dup].copy()
print("Filas:", len(df))
muestra                                    # revísalos ANTES de borrar
```

```
Filas: 3120 | duplicados exactos: 90
Duplicados por documento: 30
Filas: 3000
```

	id_afiliado	numero_documento	nombres	apellidos
1346	AF102711	1005436733	Beatriz Paola	Ramírez Pérez
2686	AF900011	1005436733	Beatriz Paola	Ramírez Pérez
671	AF900006	1027090774	Natalia	Rodríguez Peña
2842	AF102685	1027090774	Natalia	Rodríguez Peña

Qué observar

- `duplicated()` sin argumentos: idénticas en **todas** las columnas (90).
- `subset="numero_documento"`: misma persona, otra fila (30).
- Míralos **antes** de borrar: AF102711 y AF900011 son la misma Beatriz.

El documento solo aparece repetido **después** de quitarle los puntos. El orden de los pasos importa: primero limpiar, luego deduplicar.

ean universidad

05

Unir y agregar

Columnas derivadas, merge (el BUSCARV de pandas), groupby y pivot_table.

Columnas derivadas y rangos (pd.cut)

Información nueva calculada a partir de la que ya existe

Qué es

Una columna nueva que sale de una fórmula sobre otras columnas: aporte = salario × 16 %. `pd.cut` convierte un número en una **categoría de rango** con etiqueta.

Para qué sirve

El informe necesita aporte esperado, edad y rango salarial, y el archivo no los trae. Se calculan, no se capturan.

Analogía con Excel

Escribir =C2*16% y arrastrar hasta la fila 3.000. `pd.cut` es la columna "tramo" que armarías con tres SI() anidados.

Cómo manejarlo

Escribe la fórmula con la columna completa. Para rangos: `bins` con límites claros, `labels` legibles, y decide si el límite inferior va incluido (`right=False`).

Abstracción desde el código

```
df["aporte_esperado"] =
(df["salario_base"] * 0.16).round()


df["rango"] = pd.cut(df["salario_base"],
    bins=[0, S, 2*S, 4*S, float("inf")],
    labels=["< 1", "1-2", "2-4", "> 4
SMMLV"],
    right=False)
```

No existe la "fila actual": la operación se piensa para la columna entera y pandas la aplica a todas. Ese cambio de mentalidad es la mitad de pandas.

Columnas derivadas y rangos

Nueva información calculada a partir de la existente

[9]

```python
df["aporte_esperado"] = (df["salario_base"] * 0.16).round()

# pd.cut: convierte un número en rangos
df["rango_salarial"] = pd.cut(df["salario_base"],
    bins=[0, SMMLV, 2 * SMMLV, 4 * SMMLV, float("inf")],
    labels=["< 1 SMMLV", "1-2 SMMLV", "2-4 SMMLV", "> 4 SMMLV"],
right=False)

print(df["rango_salarial"].value_counts().to_dict())
df[["salario_base", "aporte_esperado", "rango_salarial"]].head(4)
```

```
{'1-2 SMMLV': 1742, '2-4 SMMLV': 981, '> 4 SMMLV': 172, '< 1 SMMLV': 60}
```

	salario_base	aporte_esperado	rango_salarial
0	1819000.0	291040.0	1-2 SMMLV
1	1760000.0	281600.0	1-2 SMMLV
2	8985000.0	1437600.0	> 4 SMMLV
3	3549000.0	567840.0	2-4 SMMLV

Qué observar

- `aporte_esperado` = salario × 16 %, en una línea para 3.000 filas.
- `pd.cut` convierte un número en **rangos** con etiquetas.
- `right=False`: el límite inferior incluido, el superior no.

Los 60 afiliados con salario **menor a 1 SMMLV** no son un error de limpieza: son un hallazgo de negocio. Mañana los revisamos con una regla.

Unir tablas: merge

Traer columnas de otra tabla usando una llave en común

Qué es

Une dos tablas por una columna compartida (la **llave**) y trae las columnas de la segunda a la primera. `how` dice qué pasa con las filas que no encuentran pareja.

Para qué sirve

Traer la regional y el código DANE del catálogo de oficinas a cada afiliado, para poder agrupar por regional.

Analogía con Excel

BUSCARV: la llave es el valor buscado, `how="left"` es "conserva todas mis filas", y `_merge` es el #N/A que Excel no te cuenta.

Cómo manejarlo

Limpia la llave en **ambas** tablas. Usa `how="left"` e `indicator=True`. Cuenta los `left_only`: cada uno es una fila sin pareja que debes explicar.

Abstracción desde el código

```
cat = pd.read_excel("oficinas.xlsx",
sheet_name="ciudades")


df = df.merge(cat[["ciudad", "regional"]],
              on="ciudad", how="left",
              indicator=True)


df["_merge"].value_counts()   # both /
left_only
```

Un cruce es una pregunta: "¿cada afiliado encuentra su ciudad en el catálogo?". `indicator=True` convierte la pregunta en un conteo que puedes reportar.

Unir tablas: merge es el BUSCARV de pandas

Traer la regional desde el catálogo oficinas.xlsx

[10]

```
# Con la ciudad SIN limpiar, ¿cuántos se quedan sin regional?
crudo = afiliados_raw.merge(cat[["ciudad", "regional"]], on="ciudad",
how="left", indicator=True)
print("Sin limpiar:", (crudo["_merge"] == "left_only").sum(), "afiliados sin
regional")

# Con la ciudad limpia
df = df.merge(cat[["ciudad", "codigo_dane", "regional"]],
              on="ciudad", how="left", indicator=True)
print(df["_merge"].value_counts().to_dict())
df = df.drop(columns="_merge")
df[["id_afiliado", "ciudad", "codigo_dane", "regional"]].head(4)
```

```
Sin limpiar: 285 afiliados sin regional
{'both': 3000, 'left_only': 0, 'right_only': 0}
```

	id_afiliado	ciudad	codigo_dane	regional
0	AF100026	Medellín	05001	Antioquia
1	AF100492	Cartagena	13001	Caribe
2	AF101579	Manizales	17001	Eje Cafetero
3	AF102295	Bogotá D.C.	11001	Centro

Qué observar

- `on="ciudad"`: la llave. `how="left"`: conserva **todos** los afiliados.
- `indicator=True` agrega `_merge`: both = encontró pareja.
- Con la ciudad sin limpiar, **285** se quedan sin regional. Limpia, luego cruza.

Un BUSCARV que no encuentra devuelve #N/A y nadie se entera. Aquí `_merge` te lo dice en una línea: **both: 3000, left_only: 0**.

Agrupar y resumir: groupby y pivot_table

Separar, aplicar, combinar: la tabla dinámica escrita

Qué es

Partir la tabla por los valores de una columna, calcular algo en cada parte y volver a juntar (**split, apply, combine**). `pivot_table` hace lo mismo con filas y columnas.

Para qué sirve

Responder "¿cuánto por cada regional?" y "¿cuántos activos, inactivos y pensionados por regional?" en una línea cada una.

Analogía con Excel

La tabla dinámica: `groupby` es arrastrar un campo a Filas; `pivot_table` es Filas + Columnas + Valores con el botón de totales (`margins`).

Cómo manejarlo

Decide la pregunta antes del código: qué agrupa (`index`), qué se cuenta (`values`), con qué función (`aggfunc`). Limpia primero: cada variante de "Activo" sería una columna más.

Abstracción desde el código

```
df.groupby("regional").agg(
  afiliados=("id_afiliado", "count"),
  mediana=("salario_base", "median"))

df.pivot_table(index="regional",
  columns="estado", values="id_afiliado",
  aggfunc="count", margins=True)
```

Una vez escrita, la tabla dinámica se regenera con el archivo del mes que viene sin un solo clic. Esa es la diferencia entre hacer un reporte y tener un reporte.

Agrupar y resumir: groupby + agg

La pregunta "¿cuánto por cada…?"

[11]

```
resumen = (df.groupby("regional")
             .agg(afiliados=("id_afiliado", "count"),
                  salario_promedio=("salario_base", "mean"),
                  edad_promedio=("edad", "mean"))
             .round(0)
             .sort_values("afiliados", ascending=False))
resumen
```

regional	afiliados	salario_promedio	edad_promedio
Centro	1247	3446475.0	45.0
Caribe	481	3539830.0	46.0
Occidente	415	3509774.0	44.0
Antioquia	391	3745402.0	44.0
Oriente	274	3473602.0	44.0
Eje Cafetero	192	3431539.0	44.0

Qué observar

- `groupby("regional")`: una fila por regional.
- `agg(nombre=("columna", "función"))`: count, sum, mean, median, min, max, nunique.
- `sort_values` ordena el resultado.

Es la tabla dinámica de Excel con una sola dimensión, pero **repetible**: el mes que viene corre igual con el archivo nuevo.

pivot_table: la tabla dinámica

Filas × columnas × valor

[12]

```
df.pivot_table(index="regional",          # filas
               columns="estado",          # columnas
               values="id_afiliado",      # qué se cuenta
               aggfunc="count",
               margins=True, margins_name="Total")
```

estado	Activo	Inactivo	Pensionado	Total
regional				
Antioquia	253	78	60	391
Caribe	292	96	93	481
Centro	774	228	245	1247
Eje Cafetero	132	30	30	192
Occidente	263	74	78	415
Oriente	171	51	52	274
Total	1885	557	558	3000

Qué observar

- `index` = filas, `columns` = columnas, `values` = qué se cuenta.
- `margins=True` agrega los totales.
- Solo 3 columnas de estado, porque ya está limpio.

Antes de limpiar esta tabla tenía **8 columnas de estado** ("ACTIVO", "activo ", "Pensionada"...). La limpieza es lo que hace posible el reporte.

De Excel a pandas: equivalencias de hoy

Lo que ya sabes hacer, ahora repetible

En Excel	En pandas
ESPACIOS(), MINUSC(), NOMPROPIO()	`.str.strip()`, `.str.lower()`, `.str.title()`
Buscar y reemplazar	`.str.replace()`, `.replace({...})`, `.map({...})`
Quitar duplicados	`drop_duplicates()` / `drop_duplicates(subset=...)`
Texto en columnas / VALOR() / FECHA()	`pd.to_numeric()`, `pd.to_datetime()`
BUSCARV / BUSCARX	`merge(..., on="llave", how="left")`
Tabla dinámica	`pivot_table()` / `groupby().agg()`
Guardar como CSV	`to_csv("archivo.csv", index=False)`

La diferencia no es la función: es que en pandas **queda escrito** y se vuelve a ejecutar con el archivo del mes siguiente.

Antes y después

Lo que vas a lograr en el laboratorio

[13]

```
pd.DataFrame({
    "antes":   [len(afiliados_raw), afiliados_raw["ciudad"].nunique(),
                afiliados_raw["estado"].nunique(),
afiliados_raw["genero"].nunique(),
                afiliados_raw["salario_base"].dtype, 285],
    "después": [len(df), df["ciudad"].nunique(), df["estado"].nunique(),
                df["genero"].nunique(), df["salario_base"].dtype, 0],
}, index=["filas", "ciudades distintas", "estados distintos", "géneros
distintos",
          "tipo de salario_base", "sin regional al cruzar"])
```

	antes	después
filas	3120	3000
ciudades distintas	39	15
estados distintos	8	3
géneros distintos	10	3
tipo de salario_base	str	float64
sin regional al cruzar	285	0

Qué observar

- 3.120 filas: −90 exactas, −30 por documento = **3.000**.
- 39 ciudades, 15. 8 estados, 3. 10 géneros, 3 (con "Sin dato").
- Salario ya es número; nadie queda sin regional.

El resultado se exporta a **afiliados_limpios.csv**: la materia prima de la Sesión 3, donde preguntamos ¿puedo confiar en estos datos?

LAB

60 min

Laboratorio · Sesión 2

Abre **PY-EAN_S02_Lab_alumno.ipynb** desde GitHub → **Open in Colab** → **Copiar en Drive**.

Limpias `afiliados.csv` paso a paso, lo unes con el catálogo de oficinas, resumes por regional y exportas `afiliados_limpios.csv`.

Checkpoints: Parte 2 a las 09:10 y Parte 3 (**3.000 filas**) a las 09:25.

Lo que te llevas hoy

Y el reto para mañana

Sobre una copia, con código

Cada corrección es una línea documentada que se vuelve a ejecutar.

Texto y categorías

strip, lower, title, quitar tildes; diccionarios de mapeo con map.

Tipos correctos

Códigos como texto; salarios con to_numeric; fechas con to_datetime.

Nulos y duplicados

Se marcan, no se inventan; exactos y por llave de negocio.

Unir y resumir

merge = BUSCARV; pivot_table = tabla dinámica; groupby + agg.

Reto para mañana

Con `pqrs.csv`: calcula los **días de respuesta** de cada solicitud y el promedio por tipo.

Vas a encontrar días **negativos** y solicitudes cerradas **sin fecha de respuesta**. Anótalos: mañana preguntamos ¿puedo confiar en estos datos? (Sesión 3 · Calidad de datos).

ean universidad

acreditada
en alta calidad
res. Nº 023654 del Mineducación
10/12/21, vigencia 10/12/27



www.universidadean.edu.co

Universidad Ean: SNIES 2812 Personería Jurídica Res. 2898 del Minjusticia - 16/05/69 - Vigilada Mineducación.

Ean Legacy: Carrera 11 No. 78-47 - Fundadores: Calle 79 # 11 - 45 - Av. Chile: Calle 71 # 9 - 84 Bogotá D.C., Cundinamarca, Colombia, Suramérica.

Línea gratuita nacional: +57-01-8000-931000
Centro de contacto en Bogotá: +(57-601) 593 6161 - +(57-601) 593 6464
E-mail: informacion@universidadean.edu.co

© 2024 Todos los derechos reservados - Universidad Ean